Zellic

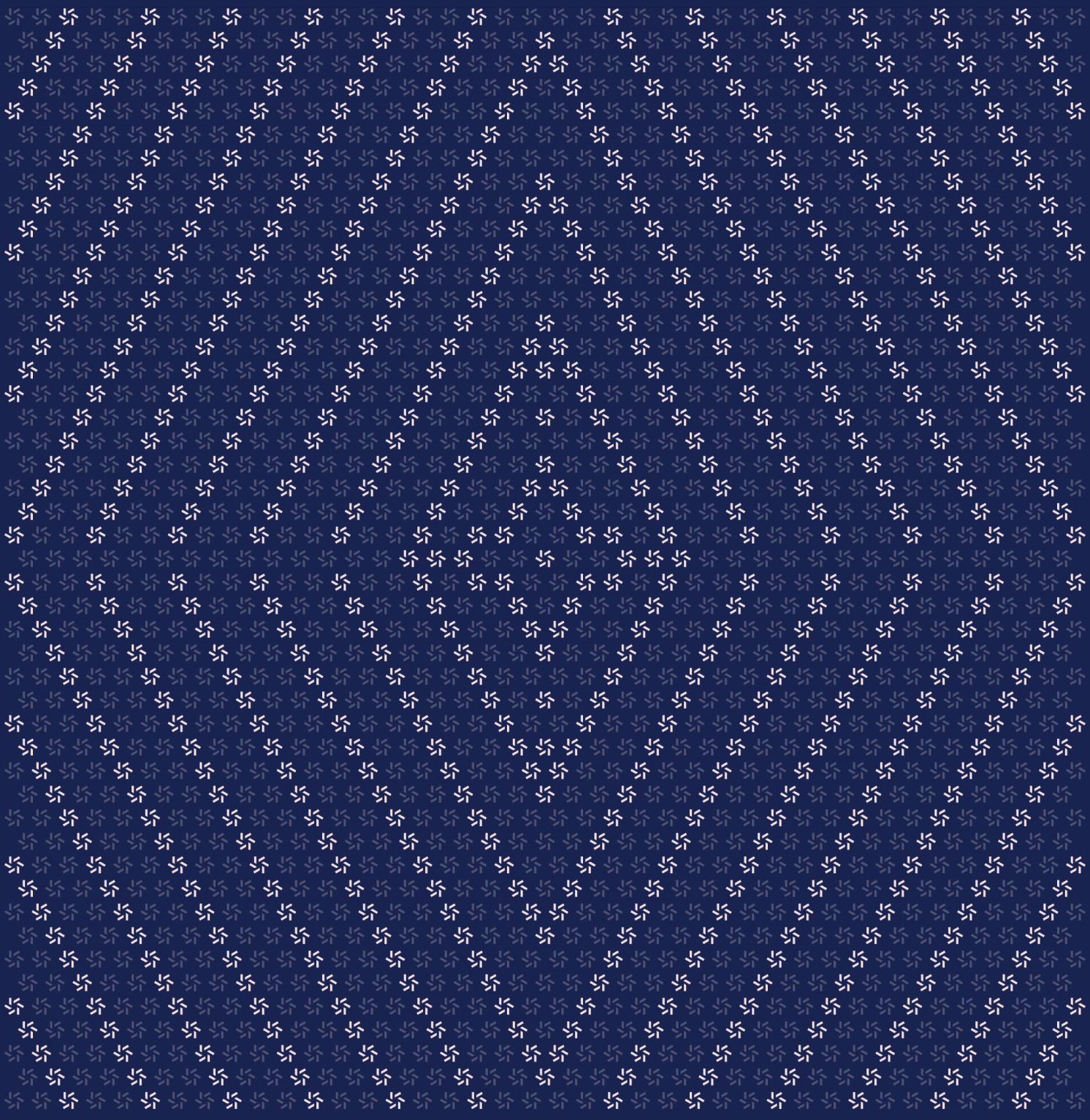
Prepared for
Jake Goh Si Yuan
Jon Walch
Superstate Inc.

Prepared by
Nipun Gupta
Xiaochuan Yu
Zellic

February 17, 2026

Superstate Smart Contracts

Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Superstate Inc. from February 9th to February 17th, 2026. During this engagement, Zellic reviewed the Superstate smart contracts' code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Are there any issues that allow circumventing the allowlist?
- Are the oracles correctly configured? Could they return inaccurate prices?
- Could a user extract more value than expected?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

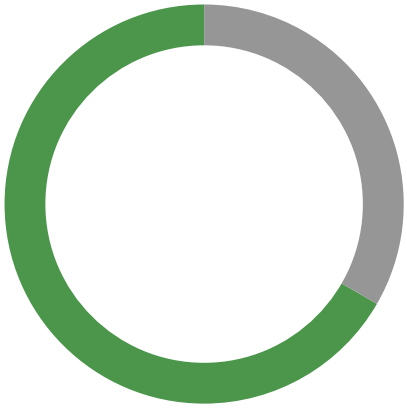
1.4. Results

During our assessment on the scoped Superstate smart contracts, we discovered six findings. No critical issues were found. Four findings were of low impact, and the remaining two findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Superstate Inc. in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	4
Informational	2



2. Introduction

2.1. About Superstate Smart Contracts

Superstate Inc. contributed the following description of Superstate Smart Contracts:

Superstate is a tokenized financial instruments platform managing on-chain Treasury Bills (USTB), Crypto Carry Fund (USCC), and equity tokens. This audit covers five contract systems: a unified Allowlist for KYC/AML compliance, two upgradeable token implementations (FundToken and EquityToken) sharing a modular component architecture, an on-chain Redemption system with oracle-priced USDC payouts (idle and yield-bearing variants), and the DIP (Direct Issuance Protocol) for discounted equity issuance via Pyth oracle pricing. All contracts are upgradeable (UUPS or initializer-based) and target Solidity 0.8.28.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

> **Basic coding mistakes.** Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.
>
> **Business logic errors.** Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.
>
> **Integration risks.** Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.
>
> **Code maturity.** We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no

hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Superstate Smart Contracts

Type	solidity
Platform	EVM-compatible
Target	superstate-evm-audit
Repository	https://github.com/superstateinc/superstate-evm-audit ↗
Version	`fcc637fc965852535bc464cfd6599589d9f76ec7`

Programs	`allowlist/src/v4.0/AllowlistV4_0.sol`
	`token/src/fund/FundToken.sol`
	`token/src/components/SuperstateTokenCore.sol`
	`token/src/components/Permittable.sol`
	`token/src/components/Allowlistable.sol`
	`token/src/components/Bridgeable.sol`
	`token/src/components/Redeemable.sol`
	`token/src/components/Subscribable.sol`
	`token/src/components/OracleConsumable.sol`
	`token/src/equity/EquityToken.sol`
	`token/src/components/Dippable.sol`
	`redemptions/src/v2/RedemptionV2.sol`
	`redemptions/src/v2/RedemptionIdleV2.sol`
	`redemptions/src/v2/RedemptionYieldV2.sol`
	`redemptions/src/oracle/SuperstateOracle.sol`
	`dip/src/Dip.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 2.1 person-weeks. The assessment was conducted by two consultants over the course of 1.4 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Nipun Gupta
Engineer
nipun@zellic.io ↗

Xiaochuan Yu
Engineer
xiaochuan@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

February 9, 2026	Start of primary review period
February 10, 2026	Kick-off call
February 17, 2026	End of primary review period

3. Detailed Findings

3.1. Dip does not disable `renounceOwnership()`

Target	Dip		
Category	Protocol Risks	**Severity**	High
Likelihood	Low	**Impact**	Low

Description

Across the system, contracts override `renounceOwnership()` to always revert (e.g., via a `RenounceOwnershipDisabled()` error or a simple revert) and explicitly document this as a critical safety feature to prevent permanent loss of admin control:

```
/**
 * @custom:security Critical safety feature to prevent permanent loss of admin
    control
 * @custom:throws RenounceOwnershipDisabled always
 */
function renounceOwnership() public virtual override {
    _checkOwner();
    revert RenounceOwnershipDisabled();
}
```

However, Dip.sol does not appear to include this override, which means the inherited `Ownable.renounceOwnership()` behavior may remain available. In OpenZeppelin's `Ownable`, `renounceOwnership()` sets the owner to `address(0)`, permanently removing privileged access.

Impact

If `renounceOwnership()` can be called on Dip, the owner can be set to `address(0)` and admin-only functionality may become permanently inaccessible.

Recommendations

We recommend adding the same `renounceOwnership()` override used elsewhere (revert unconditionally after `_checkOwner()`) in the Dip contract as well.

Remediation

This issue has been acknowledged by Superstate Inc., and a fix was implemented in commit `7a1d5ac1` ↗.

3.2. Pyth confidence interval is not validated

Target	Dip.sol		
Category	Integration Risks	**Severity**	Medium
Likelihood	Low	**Impact**	Low

Description

In Dip.sol, `_getDiscountedPrice` consumes a Pyth price-feed update but does not validate the feed's reported `conf` (confidence interval) value, which represents the uncertainty range in the reported price.

Pyth updates include both a price and a confidence interval. If the confidence interval is large, the price is less reliable and may be unsuitable for sensitive calculations.

Impact

If the Pyth feed returns an update with an unusually high confidence interval, the system may still accept it and proceed with discount calculations using a low-quality price. This can reduce pricing robustness and may cause inaccurate discounted pricing during volatile or degraded oracle conditions.

Recommendations

The official documentation ↗ of the Pyth price feeds recommends some ways in which this confidence interval can be used. For example, the value interval/price could be used to decide the level of the price's uncertainty and disallow user interaction with the system in case this value exceeds some threshold.

Remediation

This issue has been acknowledged by Superstate Inc... Additionally, the Superstate Inc. team noted that:

> The existing multi-layer defense (price clamps, staleness checks, slippage protection, owner pause) is sufficient for this protocol's risk profile, and empirical data shows that a confidence interval check would be operationally harmful for equity feeds. We also want to be consistent with the current Solana implementation which makes the same decision. This is something we will continue to monitor and implement and should the case arises that this is necessary due

to massive volatility and/or unreliable oracles.

3.3. Calling `subscribe()` with a sufficiently small `amount` can reduce the fee

Target	Subscribable.sol		
Category	Business Logic	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

First, `subscribable.subscribe()` will call `calculateSuperstateTokenOut()` to calculate the amount of tokens to mint for a subscription for the given `amount`, and it will then call `calculateFee()` to calculate the fee:

```
function calculateFee(uint256 amount, uint256 subscriptionFee)
    public pure virtual returns (uint256) {
    return (amount * subscriptionFee) / FEE_DENOMINATOR;
}
```

The `FEE_DENOMINATOR` is set to `10_000` and the maximum `subscriptionFee` is `10`, which is checked in `_setStablecoinConfig()`. For a sufficiently small `amount`, the fee will be rounded down to `0` and the subscriber will not pay any fee.

Impact

The issue is that it is normally user triggerable under standard configuration (the stable coin configuration). A user can call `subscribe()` with a small `amount` to avoid paying the fee. The same stablecoin input can mint more fund tokens when split into multiple calls.

Recommendations

Keep floor rounding, but add a minimum fee policy for nonzero configured fees; accumulate the fee rounding remainder and charge it on later calls; or set a minimum `amount` for subscription.

Remediation

This issue has been acknowledged by Superstate Inc., and a fix was implemented in commit `62977fb1` ↗.

3.4. Calling `redeem()` with a sufficiently small `superstateTokenInAmount` can reduce the fee

Target	RedemptionIdleV2.sol		
Category	Business Logic	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

First, `RedemptionIdleV2.redeem()` will call `calculateUsdcOut()` to calculate the amount of USDC to receive for the given `superstateTokenInAmount`, and it will then call `calculateFee()` to calculate the fee:

```
function calculateFee(uint256 amount) public view returns (uint256) {
    return (amount * redemptionFee) / FEE_DENOMINATOR;
}
```

The `FEE_DENOMINATOR` is set to `10_000` and the maximum `redemptionFee` is `10`, which is checked in `_setRedemptionFee()`. For a sufficiently small `amount`, the fee will be rounded down to `0` and the subscriber will not pay any fee.

Impact

The issue is that it is normally user triggerable under standard configuration (the redemption fee). A user can call `redeem()` with a small `superstateTokenInAmount` to avoid paying the fee. The same superstate token input can redeem more USDC when split into multiple calls.

Recommendations

Keep floor rounding, but add a minimum fee policy for nonzero configured fees; accumulate the fee rounding remainder and charge it on later calls; or set a minimum `superstateTokenInAmount` for redemption.

Remediation

This issue has been acknowledged by Superstate Inc., and a fix was implemented in commit `0fa60d66` ↗.

3.5. Incorrect comment might lead to incorrect assumptions and potential storage collisions

Target	FundToken		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

In `FundToken.initializeV1_0_0`, there is the following comment:

```
__Permittable_init(); // Nonces remain in continuous storage
```

This comment is inaccurate if the upgrade changes nonce storage to an EIP-7201–style layout. In such a case, nonce values would not "remain in continuous storage" across upgrades in the way the comment implies.

This can lead to confusion during review and maintenance, and it can cause incorrect assumptions about replay protection for historical signatures.

Based on the current implementation, replay can still be prevented due to a different `VERSION` in `DOMAIN_TYPEHASH` (producing a different domain separator / digest), which would make older signatures invalid under the new domain. If that assumption changes (e.g., the domain does not change or signatures are validated against the prior domain), the incorrect nonce continuity assumption could become security-relevant.

Impact

The immediate impact is documentation/maintenance risk; developers and auditors may incorrectly assume nonce continuity across upgrades.

If domain separation does not change as expected, there might be a risk of signature replay.

Recommendations

We recommend updating the comment to accurately describe nonce storage behavior across this upgrade.

Remediation

This issue has been acknowledged by Superstate Inc., and a fix was implemented in commit `4b5a630f` ↗.

3.6. Incorrect comment might lead to incorrect usage of `chainId` in `Bridgeable.bridge`

Target	Bridgeable		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

Documentation and comments in Bridgeable and `IBridgeable` state that unsupported destination chain IDs can "burn to book entry".

```
/**
 * @notice Burns tokens from the caller to bridge to another chain
 * @dev If destination address on chainId isn't on allowlist, or chainID isn't
    supported, tokens burn to book entry.
 * @dev chainId as 0 indicates wanting to burn tokens to book entry, for use
    through the Superstate UI.
 * @param amount Amount of tokens to burn
 * @param ethDestinationAddress ETH address to send to on another chain
 * @param otherDestinationAddress Non-EVM addresses to send to on another chain
 * @param chainId Numerical identifier of destination chain to send tokens to
 */
function bridge(
    uint256 amount,
    address ethDestinationAddress,
    string memory otherDestinationAddress,
    uint256 chainId
) public virtual {
    // [...]

    if (!isChainIdSupported(chainId))
    revert BridgeChainIdDestinationNotSupported();

    _burn(msg.sender, amount);
    // [...]
}
```

The actual implementation behavior is different; if `chainId` is unsupported, `bridge(...)` reverts with `BridgeChainIdDestinationNotSupported()` and no fallback burn is performed.

Impact

Developers and maintainers may expect graceful fallback but receive hard reverts and failed user transactions. This can lead to confusion in usage and maintenance.

Recommendations

We recommend to update the comments in Bridgeable and `IBridgeable` to accurately reflect the behavior of unsupported destination chain IDs.

Remediation

This issue has been acknowledged by Superstate Inc., and a fix was implemented in commit 35e30708 ↗.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. The `redemptionFee` is assigned by `initialize()` directly, bypassing the 10 bps cap in `_setRedemptionFee()`

RedemptionV2 enforces an explicit maximum fee in `_setRedemptionFee()` (reverting if `_newFee > 10`). However, `initialize()` sets `redemptionFee = _redemptionFee` directly and emits `SetRedemptionFee`, which bypasses the cap check that exists for later updates, as shown below:

```
function initialize(
    address initialOwner,
    uint256 _maximumOracleDelay,
    address _sweepDestination,
    uint256 _redemptionFee
) external initializer {
    __Ownable_init(initialOwner);
    __Ownable2Step_init();

    _setMaximumOracleDelay(_maximumOracleDelay);
    _setSweepDestination(_sweepDestination);

    redemptionFee = _redemptionFee;
    emit SetRedemptionFee({oldFee: 0, newFee: _redemptionFee});
}

function _setRedemptionFee(uint256 _newFee) internal {
    if (_newFee > 10) revert FeeTooHigh(); // Max 0.1% fee
    if (redemptionFee == _newFee) revert BadArgs();
    emit SetRedemptionFee({oldFee: redemptionFee, newFee: _newFee});
    redemptionFee = _newFee;
}
```

We recommend adding the same cap in initialize as well.

This issue has been acknowledged by Superstate Inc., and a fix was implemented in commit `0fa60d66` ↗.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: AllowlistV4_0.sol

Function: `setAllowedWithSignature(address addr, address token, bool allowed, uint256 nonce, uint256 deadline, uint8 v, byte[32] r, byte[32] s)`

This function sets the permission for an address for a token via EIP-712 signature.

Inputs

- `addr`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must match the signed payload — zero checked in `_setAllowed`.
  - **Impact**: Target of permission update.
- `token`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must match the signed payload.
  - **Impact**: Used in allowed[token][addr].
- `allowed`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must match the signed payload.
  - **Impact**: Value written to storage.
- `nonce`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must equal `$.nonces[addr]` — then incremented.
  - **Impact**: Replay protection.
- `deadline`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be `>= block.timestamp`.
  - **Impact**: Expired signatures revert.
- `v`

    - **Control**: Controllable by the caller.
    - **Constraints**: Part of signature — must recover to `owner()`.
    - **Impact**: Wrong signer reverts.
- `r`
    - **Control**: Controllable by the caller.
    - **Constraints**: Part of signature — must recover to `owner()`.
    - **Impact**: Wrong signer reverts.
- `s`
    - **Control**: Controllable by the caller.
    - **Constraints**: Part of signature — must recover to `owner()`.
    - **Impact**: Wrong signer reverts.

Branches and code coverage

Intended branches

- `allowed` is set for the address and token.
    - ☑ Test coverage

Negative behavior

- Expired signature (deadline passed).
    - ☑ Negative test
- Invalid nonce (replay).
    - ☑ Negative test
- Invalid signer (recover does not equal the owner).
    - ☑ Negative test

Function call analysis

- `ECDSA.recover(digest, v, r, s)`
    - **What is controllable?** `digest`, `v`, `r`, and `s` (caller).
    - **If the return value is controllable, how is it used and how can it go wrong?** Compared to `owner()` — mismatch reverts `InvalidSigner`.
    - **What happens if it reverts, reenters or does other unusual control flow?** Pure — no reentrancy.
- `this._setAllowed(addr, token, allowed)`
    - **What is controllable?** `addr`, `token`, and `allowed` (from signed payload).
    - **If the return value is controllable, how is it used and how can it go wrong?**

No return — writes storage. Zero `addr` or `AlreadySet` reverts.

- **What happens if it reverts, reenters or does other unusual control flow?** No external calls.

5.2. Module: Bridgeable.sol

Function: `bridge(uint256 amount, address ethDestinationAddress, string otherDestinationAddress, uint256 chainId)`

This function burns tokens from the caller to bridge to another chain.

Inputs

- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of tokens to burn.
- `ethDestinationAddress`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The ETH address to send to, on another chain.
- `otherDestinationAddress`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The address to send to, on another chain.
- `chainId`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The chain ID to send tokens to.

Branches and code coverage

Intended branches

- Burn the tokens and emit the `Bridge` event.
  - ☑ Test coverage

Negative behavior

- Revert if the amount is zero.

☑ Negative test

- Revert if the caller is not allowed.

☑ Negative test

- Revert if the contract is paused.

☑ Negative test

- Revert if the chain ID is not supported.

☑ Negative test

- Revert if the ETH and other destination addresses are both set.

☑ Negative test

- Revert if the chain ID is zero and any one of the ETH and other destination addresses is set.

☑ Negative test

Function: `bridgeToBookEntry(uint256 amount)`

This function burns tokens from the caller to bridge to the Superstate book entry.

Inputs

- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of tokens to burn.

Branches and code coverage

Intended branches

- Burn the tokens and emit the `Bridge` event.

☑ Test coverage

Negative behavior

- Revert if the amount is zero.

☐ Negative test

- Revert if the caller is not allowed.

☐ Negative test

- Revert if the contract is paused.

- ☐ Negative test
- Revert if the chain ID is not supported.
  - ☐ Negative test
- Revert if the ETH and other destination addresses are both set.
  - ☐ Negative test

Function call analysis

- `this.bridge(amount, address(0), string(new bytes(0)), 0)`
  - **What is controllable?** `amount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** If the inner call reverts, the function will revert; no reentrancy is possible.

5.3. Module: Dip.sol

Function: `buyTheDip(byte[32] marketId, address buyer, uint256 paymentAmount, uint256 minOutAmount, IERC20 paymentToken)`

This function executes a direct instrument purchase with partial fill support.

Inputs

- `marketId`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be a valid market ID.
  - **Impact**: Market to purchase from.
- `buyer`
  - **Control**: Controllable by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Buyer address.
- `paymentAmount`
  - **Control**: Controllable by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Payment amount.
- `minOutAmount`

  - **Control**: Controllable by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Minimum output amount.
- `paymentToken`
  - **Control**: Controllable by the caller.
  - **Constraints**: Should be the same as the market's payment token.
  - **Impact**: Payment token.

Branches and code coverage

Intended branches

- If the remaining capacity is less than the minimum payment, the market is set to `Closed`.
  - ☑ Test coverage
- Return the actual payment amount, instrument amount, and recipient.
  - ☑ Test coverage

Negative behavior

- If the caller is not the current market's instrument token, revert with `NotCurrentMarket`.
  - ☑ Negative test
- If the payment token does not match the market's payment token, revert with `NotPaymentToken`.
  - ☑ Negative test
- If the instrument amount is less than the minimum output amount, revert with `InsufficientOutput`.
  - ☑ Negative test
- If the actual payment amount is zero, revert with `ZeroActualPayment`.
  - ☑ Negative test
- If the price feed exceeds the max latency, revert with `PriceFeedExceedsMaxLatency`.
  - ☑ Negative test
- If the price feed is invalid, revert with `PriceFeedInvalidOutput`.
  - ☑ Negative test
- If the price is out of bounds, revert with `PriceOutOfBounds`.
  - ☑ Negative test

Function call analysis

- `this._calculateOutput(marketId, paymentAmount, paymentDecimals)`
  - **What is controllable?** `marketId`, `paymentAmount`, and `paymentDecimals`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls.
- `this._setMarketState(market, State.Closed)`
  - **What is controllable?** `market`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return — writes storage.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls.

5.4. Module: Dippable.sol

Function: `buyTheDip(byte[32] marketId, uint256 paymentAmount, uint256 minOutAmount, IERC20 paymentToken)`

This function purchases tokens through a DIP market with oracle-based discounted pricing.

Inputs

- `marketId`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The market ID to purchase from.
- `paymentAmount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of payment tokens to spend.
- `minOutAmount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The minimum amount of tokens to receive (slippage protection, validated by DIP).
- `paymentToken`
  - **Control**: Fully controlled by the caller.

- **Constraints**: No constraints.
- **Impact**: The token to use for payment (must match market's payment token).

Branches and code coverage

Intended branches

- Call the `buyTheDip` function of the Dip contract.
  - ☑ Test coverage
- Transfer the payment from the caller to the recipient.
  - ☑ Test coverage
- Mint the tokens to the caller.
  - ☑ Test coverage

Negative behavior

- Revert if the payment amount is zero.
  - ☑ Negative test
- Revert if the minimum amount of tokens to receive is zero.
  - ☑ Negative test
- Revert if the Dip contract is not set.
  - ☑ Negative test
- Revert if the contract is paused.
  - ☑ Negative test
- Revert if the payment token does not match the market's payment token (inside the `buyTheDip` function).
  - ☐ Negative test
- Revert if the caller is not the current market's instrument token (inside the `buyTheDip` function).
  - ☑ Negative test

Function call analysis

- `IDip(dip).buyTheDip(marketId, msg.sender, paymentAmount, minOutAmount, paymentToken)`
  - **What is controllable?** `marketId`, `msg.sender`, `paymentAmount`, `minOutAmount`, and `paymentToken`.

- **If the return value is controllable, how is it used and how can it go wrong?** `actualPaymentAmount`, `instrumentAmount`, and `paymentRecipient` are returned. The `paymentRecipient` it sent the `actualPaymentAmount` amount of paymentTokens and then the function mints `instrumentAmount` amounts of token to the msg.sender.
- **What happens if it reverts, reenters or does other unusual control flow?** If the inner call reverts, the function will revert; no reentrancy is possible.

- `SafeERC20.safeTransferFrom(paymentToken, msg.sender, paymentRecipient, actualPaymentAmount)`
  - **What is controllable?** `paymentToken`, `msg.sender`, `paymentRecipient`, and `actualPaymentAmount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** If the inner call reverts, the function will revert; no reentrancy is possible.

5.5. Module: EquityToken.sol

Function: `transfer(address dst, uint256 amount)`

This function moves tokens from the caller to destination address.

Inputs

- `dst`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be allowlisted.
  - **Impact**: Can be used to transfer tokens to any address.
- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Can be used to transfer any amount of tokens.

Branches and code coverage

Intended branches

- If the `dst` is the contract address, the function will burn the tokens and emit the `Bridge` event.
  - ☑ Test coverage

- If the `dst` is not the contract address, the function will transfer the tokens to the `dst` address.

  ☑ Test coverage

Negative behavior

- Revert if the caller is not allowlisted.

  ☑ Negative test
- Revert if `dst` is not allowlisted.

  ☑ Negative test
- Revert if the contract is paused.

  ☑ Negative test

Function call analysis

- `this._requireAllowed(src) -> this.isAllowed(addr) -> IAllowlistAddressPermissions(_allowlist).isAddressAllowedForPublicInstrument(addr)`
  - **What is controllable?** `src` address.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls inside.

Function: `transferFrom(address src, address dst, uint256 amount)`

This function moves tokens from source to destination using the caller's allowance.

Inputs

- `src`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be allowlisted.
  - **Impact**: Can be used to transfer tokens from any address to the caller.
- `dst`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be allowlisted.
  - **Impact**: Can be used to transfer tokens to any address.
- `amount`

- **Control**: Fully controlled by the caller.
- **Constraints**: No constraints.
- **Impact**: Can be used to transfer any amount of tokens.

Branches and code coverage

Intended branches

- If the `dst` is the contract address, the function will burn the tokens and emit the `Bridge` event.
  - ☑ Test coverage
- If the `dst` is not the contract address, the function will transfer the tokens to the `dst` address.
  - ☑ Test coverage

Negative behavior

- Revert if `src` is not allowlisted.
  - ☑ Negative test
- If the amount is zero, the function will revert.
  - ☐ Negative test
- If the caller does not have enough allowance, the function will revert.
  - ☐ Negative test
- If the contract is paused, the function will revert.
  - ☑ Negative test

Function call analysis

- `this._requireAllowed(src) -> this.isAllowed(addr) -> IAllowlistAddressPermissions(_allowlist).isAddressAllowedForPublicInstrument(addr)`
  - **What is controllable?** `src` address.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls inside.

5.6. Module: FundToken.sol

Function: `transfer(address dst, uint256 amount)`

This function moves tokens from the caller to destination address.

Inputs

- `dst`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: Must be allowlisted.
  - **Impact**: Can be used to transfer tokens to any address.
- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Can be used to transfer any amount of tokens.

Branches and code coverage

Intended branches

- If the `dst` is the contract address, the function will burn the tokens and emit the `OffchainRedeem` event.
  - ☑ Test coverage
- If the `dst` is not the contract address, the function will transfer the tokens to the `dst` address.
  - ☑ Test coverage

Negative behavior

- Revert if `src` is not allowlisted.
  - ☑ Negative test
- Revert if `dst` is not allowlisted.
  - ☑ Negative test
- Revert if the caller does not have enough allowance.
  - ☐ Negative test
- If the contract is paused, the function will revert.
  - ☑ Negative test

Function call analysis

- `this.isAllowed(src) -> this.isAllowed(addr) -> IAllowlistAddressPermissions(_allowlist).isAddressAllowedForPublicInstrument(addr)`
    - **What is controllable?** `src` address.
    - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
    - **What happens if it reverts, reenters or does other unusual control flow?** No external calls inside.

Function: `transferFrom(address src, address dst, uint256 amount)`

This function moves tokens from source to destination using the caller's allowance.

Inputs

- `src`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must be allowlisted.
    - **Impact**: Can be used to transfer tokens from any address to the caller.
- `dst`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: Must be allowlisted.
    - **Impact**: Can be used to transfer tokens to any address.
- `amount`
    - **Control**: Fully controlled by the caller.
    - **Constraints**: No constraints.
    - **Impact**: Can be used to transfer any amount of tokens.

Branches and code coverage

Intended branches

- If the `dst` is the contract address, the function will burn the tokens and emit the `OffchainRedeem` event.
    - ☑ Test coverage
- If the `dst` is not the contract address, the function will transfer the tokens to the `dst` address.
    - ☑ Test coverage

Negative behavior

- Revert if `src` is not allowlisted.
  - ☑ Negative test
- Revert if `dst` is not allowlisted.
  - ☑ Negative test
- Revert if the caller does not have enough allowance.
  - ☑ Negative test
- If the contract is paused, the function will revert.
  - ☑ Negative test

Function call analysis

- `this.isAllowed(src) -> this.isAllowed(addr) -> IAllowlistAddressPermissions(_allowlist).isAddressAllowedForPublicInstrument(addr)`
  - **What is controllable?** `src` address.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls inside.

5.7. Module: Permittable.sol

Function: `permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, byte[32] r, byte[32] s)`

This function sets the approval amount for a spender via a signature from the signatory.

Inputs

- `owner`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The address that signed the signature.
- `spender`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The address to authorize (or rescind authorization from).

- `value`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of tokens to approve.
- `deadline`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The deadline for the signature.
- `v`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The recovery byte of the signature.
- `r`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Half of the ECDSA signature pair.
- `s`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: Half of the ECDSA signature pair.

Branches and code coverage

Intended branches

- Increment the nonce for the owner.
  - ☑ Test coverage
- Approve the spender the amount of tokens.
  - ☑ Test coverage

Negative behavior

- Revert if the deadline is in the past.
  - ☑ Negative test
- Revert if the signature is invalid.
  - ☑ Negative test
- Revert if the nonce is invalid.
  - ☑ Negative test

Function call analysis

- `this._isValidSignature(owner, digest, v, r, s)`
  - **What is controllable?** `owner`, `digest`, `v`, `r`, and `s`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns true if the signature is valid and reverts otherwise.
  - **What happens if it reverts, reenters or does other unusual control flow?** If the inner call reverts, the function will revert; no reentrancy is possible.

5.8. Module: Redeemable.sol

Function: `offchainRedeem(uint256 amount)`

This function burns tokens from the caller's address for off-chain redemption.

Inputs

- `amount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of tokens to burn.

Branches and code coverage

Intended branches

- Burn the tokens from the caller's address.
  - ☑ Test coverage
- Emit the `OffchainRedeem` event.
  - ☑ Test coverage

Negative behavior

- Revert if the contract is paused.
  - ☑ Negative test
- Revert if the caller is not allowed.
  - ☑ Negative test

5.9. Module: RedemptionIdleV2.sol

Function: `redeem(uint256 superstateTokenInAmount)`

The redeem function allows users to redeem `SUPERSTATE_TOKEN` for USDC at the current oracle price.

Inputs

- `superstateTokenInAmount`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be a valid amount of `SUPERSTATE_TOKEN` to redeem.
  - **Impact**: Amount of `SUPERSTATE_TOKEN` to redeem.

Branches and code coverage

Intended branches

- Transfer the `SUPERSTATE_TOKEN` from the caller to the contract and return the amount of USDC received.
  - ☑ Test coverage

Negative behavior

- If the contract is paused, revert with `Paused`.
  - ☑ Negative test
- If the USDC balance of the contract is less than the amount of USDC to redeem, revert with `InsufficientBalance`.
  - ☑ Negative test
- If the `SUPERSTATE_TOKEN` balance of the contract is less than the amount of `SUPERSTATE_TOKEN` to redeem, revert with `InsufficientBalance`.
  - ☑ Negative test
- If the `SUPERSTATE_TOKEN` balance of the contract is less than the amount of `SUPERSTATE_TOKEN` to redeem, revert with `InsufficientBalance`.
  - ☑ Negative test

Function call analysis

- `SafeERC20.safeTransferFrom(this.SUPERSTATE_TOKEN, from: msg.sender, to: address(this), value: superstateTokenInAmount)`

- **What is controllable?** `msg.sender` and `superstateTokenInAmount`.
- **If the return value is controllable, how is it used and how can it go wrong?** No return — writes storage.
- **What happens if it reverts, reenters or does other unusual control flow?** No external calls.

- `SafeERC20.safeTransfer(this.USDC, to: msg.sender, value: usdcOutAmount)`
  - **What is controllable?** `msg.sender` and `usdcOutAmount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return — writes storage.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls.
- `ISuperstateToken(address(this.SUPERSTATE_TOKEN)).offchainRedeem( superstateTokenInAmount)`
  - **What is controllable?** `superstateTokenInAmount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Calls the `SUPERSTATE_TOKEN` contract's `offchainRedeem` function — no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** The `SUPERSTATE_TOKEN` contract's `offchainRedeem` function burns the tokens and emits the `OffchainRedeem` event.

5.10. Module: RedemptionYieldV2.sol

Function: `redeem(uint256 superstateTokenInAmount)`

The redeem function allows users to redeem `SUPERSTATE_TOKEN` for USDC via `COMPOUND` at the current oracle price.

Inputs

- `superstateTokenInAmount`
  - **Control**: Controllable by the caller.
  - **Constraints**: Must be a valid amount of `SUPERSTATE_TOKEN` to redeem.
  - **Impact**: Amount of `SUPERSTATE_TOKEN` to redeem.

Branches and code coverage

Intended branches

- Transfer the `SUPERSTATE_TOKEN` from the caller to the contract and return the amount of USDC received via `COMPOUND`.

- ☑ Test coverage

Negative behavior

- If the contract is paused, revert with `Paused`.
  - ☑ Negative test
- If the `COMPOUND` balance of the contract is less than the amount of USDC to redeem, revert with `InsufficientBalance`.
  - ☑ Negative test

Function call analysis

- `SafeERC20.safeTransferFrom(this.SUPERSTATE_TOKEN, from: msg.sender, to: address(this), value: superstateTokenInAmount)`
  - **What is controllable?** `msg.sender` and `superstateTokenInAmount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return — writes storage.
  - **What happens if it reverts, reenters or does other unusual control flow?** No external calls.
- `this.COMPOUND.withdrawTo(to: msg.sender, asset: address(this.USDC), amount: usdcOutAmount)`
  - **What is controllable?** `msg.sender` and `usdcOutAmount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return — writes storage.
  - **What happens if it reverts, reenters or does other unusual control flow?** Calls the `COMPOUND` contract's `withdrawTo` function — no return value.
- `ISuperstateToken(address(this.SUPERSTATE_TOKEN)).offchainRedeem(superstateTokenInAmount)`
  - **What is controllable?** `superstateTokenInAmount`.
  - **If the return value is controllable, how is it used and how can it go wrong?** Calls the `SUPERSTATE_TOKEN` contract's `offchainRedeem` function — no return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** The `SUPERSTATE_TOKEN` contract's `offchainRedeem` function burns the tokens and emits the `OffchainRedeem` event.

5.11. Module: Subscribable.sol

Function: `subscribe(address to, uint256 inAmount, address stablecoin)`

This function processes a subscription on behalf of another address.

Inputs

- `to`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The address to subscribe on behalf of.
- `inAmount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of stablecoin to subscribe with.
- `stablecoin`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The stablecoin to subscribe with.

Branches and code coverage

Intended branches

- Call the `_subscribe` function.
  - ☑ Test coverage
- Transfer the stablecoin from the caller to the sweep destination.
  - ☑ Test coverage
- Mint the tokens to the to address.
  - ☑ Test coverage

Negative behavior

- Revert if the `inAmount` is zero.
  - ☐ Negative test
- Revert if the contract is paused.
  - ☐ Negative test
- Revert if the stablecoin is not supported.
  - ☐ Negative test
- Revert if the superstate token-out amount is zero.
  - ☐ Negative test

Function call analysis

- `this._subscribe(to, inAmount, stablecoin)`
  - **What is controllable?** `to`, `inAmount`, and `stablecoin`.
  - **If the return value is controllable, how is it used and how can it go wrong?** No return value.
  - **What happens if it reverts, reenters or does other unusual control flow?** If the inner call reverts, the function will revert; no reentrancy is possible.

Function: `subscribe(uint256 inAmount, address stablecoin)`

This function processes a subscription with the caller as the recipient.

Inputs

- `inAmount`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The amount of stablecoin to subscribe with.
- `stablecoin`
  - **Control**: Fully controlled by the caller.
  - **Constraints**: No constraints.
  - **Impact**: The stablecoin to subscribe with.

Branches and code coverage

Intended branches

- Call the `_subscribe` function.
  - ☑ Test coverage
- Transfer the stablecoin from the caller to the sweep destination.
  - ☑ Test coverage
- Mint the tokens to the recipient.
  - ☑ Test coverage

Negative behavior

- Revert if the `inAmount` is zero.
  - ☑ Negative test

- Revert if the contract is paused.
  - ☑ Negative test
- Revert if the stablecoin is not supported.
  - ☑ Negative test
- Revert if the superstate token out amount is zero.
  - ☑ Negative test

6. Assessment Results

During our assessment on the scoped Superstate smart contracts, we discovered six findings. No critical issues were found. Four findings were of low impact, and the remaining two findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.